

Semana 1 – Guía de prácticas (2h)

Objetivo: Dejar operativo el entorno (VS Code + Node.js) y construir una **página base HTML5 semántica y accesible** que sirva de esqueleto para los retos posteriores.

RDAs asociados: RDA 1 (fundamentos para diseño e implementación correcta).

Requisitos previos

- **SO:** Windows/macOS/Linux
- **Software:** Navegador moderno (Chrome/Edge/Firefox), conexión a internet (solo para instalación).
- **Conocimientos:** Estructura básica de archivos, navegación por carpetas, nociones introductorias de HTML.

Bloque 0 (20 min) — Principiantes: HTML, CSS y DOM + “Hola Mundo”

0.1 ¿Qué es HTML, CSS y el DOM? (5 min)

- **HTML:** lenguaje de **marcado** → define la **estructura** y significado del contenido (títulos, párrafos, listas, imágenes, enlaces).
- **CSS:** lenguaje de **estilos** → define la **apariencia** (colores, tamaños, disposición).
- **DOM** (*Document Object Model*): representación en **árbol** del documento en memoria cuando el navegador carga el HTML. Cada etiqueta es un *nodo* (padres, hijos, hermanos).
 - **Idea clave:** el HTML es el “plano”; el DOM es el “árbol vivo” que el navegador puede modificar.

0.2 Tu primer “Hola Mundo” (8 min)

1. Crea una carpeta temporal: hola-mundo/.
2. Dentro, crea index.html con lo mínimo:

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Hola Mundo</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
</head>
<body>
  <h1>¡Hola Mundo!</h1>
  <p>Mi primera página web.</p>
</body>
</html>
```

3. Abre el archivo en el navegador (doble clic o arrástralo al navegador).
4. (Opcional) Crea styles.css:
- 5.

```
body { font-family: system-ui, sans-serif; line-height: 1.5; margin: 2rem; }
h1 { font-size: 2rem; }
```

y **enlázalo** en el <head>:

`<link rel="stylesheet" href="styles.css">`

0.3 Ver el DOM y tocarlo (7 min)

- Abre **DevTools** (Chrome/Edge: Ctrl+Shift+I, macOS: Cmd+Opt+I).
- Pestaña **Elements**: verás la estructura **DOM**.
- Haz doble clic sobre <h1> y cambia el texto → observa que el **DOM cambia en vivo**, aunque el archivo **HTML no se modifica** (si recargas, vuelve al original).
- Diferencia **Ver código fuente** (Ctrl+U) vs **Elements**: el primero muestra el HTML de archivo; el segundo, el DOM actual.

0.4 Micro-ejercicios (no calificados, 0–5 min)

- Agrega un **enlace**: Sitio PUCE (texto del enlace debe describir el destino).
- Agrega una **lista** con 3 ítems.
- Agrega una **imagen** con alt **descriptivo** (si no tienes imagen, déjala para luego).

Hora 1 (≈60 min) – Entorno listo + proyecto base

1) Verifica el entorno (5 min)

- Abre una terminal y ejecuta:
 - node -v → debería devolver una versión (LTS recomendada).
 - npm -v → debería devolver la versión de npm.
- Si alguno falla, procede a instalar.

2) Instala Node.js LTS (10–15 min)

- Descarga e instala **Node.js (LTS)** para tu sistema operativo.
- Cierra y reabre la terminal. Repite node -v y npm -v.

3) Instala VS Code + extensiones clave (10 min)

- Instala **Visual Studio Code**.
- Abre VS Code y agrega extensiones:
 - *Live Server* (vista previa local de HTML).
 - *Prettier* (formateo).
 - *ESLint* (lo usaremos más adelante).
 - *HTMLHint* (opcional, validación HTML).

4) Crea la estructura del proyecto (10 min)

- Crea una carpeta de trabajo, por ejemplo: semana1-plataformas/.
- Dentro, crea:
 - semana1-plataformas/
 - └─ public/

- | └─ index.html
- | └─ assets/
- | └─ images/ (vacío por ahora)
- | └─ styles/
- | └─ styles.css
- └─ README.md
- Abre la carpeta en VS Code (**File > Open Folder**).

5) Inicializa Git + npm (10 min)

1. Abre **PowerShell** como usuario normal y ejecuta:

```
winget install --id Git.Git -e
```

Si te pide confirmar, acepta.

2. Cierra **todas** las terminales/VS Code y vuelve a abrir.
3. Verifica:

```
git --version
```

- En la terminal integrada (View > Terminal):
 - git init
 - npm init -y (crea package.json)
- crea .gitignore
 - En VS Code, menú **File → New File**.
 - Escribe el nombre **.gitignore** (con el punto primero) y guarda en la raíz del proyecto.
 - Pega esto y guarda:

```
node_modules/
.env
.DS_Store
```

node_modules/ ignora la carpeta completa, .env ignora el archivo de variables, .DS_Store ignora el archivo de macOS.

6) Crea el esqueleto HTML semántico (10 min)

En public/index.html, pega y **ajusta** (metadatos, título, idioma):

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <title>Base – Sistema de Gestión de Procesos</title>
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <meta name="description" content="Esqueleto semántico y accesible para el curso
  Desarrollo en Plataformas." />
  <link rel="stylesheet" href="../styles/styles.css" />
</head>
```

```

<body>
  <a class="skip-link" href="#contenido">Saltar al contenido</a>

  <header>
    <h1>Desarrollo en Plataformas</h1>
    <nav aria-label="Navegación principal">
      <ul>
        <li><a href="#" aria-current="page">Inicio</a></li>
        <li><a href="#modulos">Módulos</a></li>
        <li><a href="#accesibilidad">Accesibilidad</a></li>
      </ul>
    </nav>
  </header>

  <main id="contenido">
    <section aria-labelledby="sec-intro">
      <h2 id="sec-intro">Introducción</h2>
      <p>Este es el esqueleto base para el proyecto del curso. Prioriza semántica y
accesibilidad.</p>
    </section>

    <section id="modulos" aria-labelledby="sec-modulos">
      <h2 id="sec-modulos">Módulos</h2>
      <article aria-labelledby="art-turnos">
        <h3 id="art-turnos">Módulo: Gestión de Turnos</h3>
        <p>Contenido placeholder. Aquí, más adelante, se integrará cliente-servidor.</p>
      </article>
      <aside aria-label="Información complementaria">
        <p>Nota: Mantén una jerarquía de encabezados coherente (h1→h2→h3).</p>
      </aside>
    </section>

    <section id="accesibilidad" aria-labelledby="sec-a11y">
      <h2 id="sec-a11y">Accesibilidad</h2>
      <p role="note" aria-live="polite">Recuerda: texto alternativo en imágenes y formularios
etiquetados.</p>
    </section>
  </main>

  <footer>
    <p>&copy; 2025. Curso Desarrollo en Plataformas.</p>
  </footer>
</body>
</html>

```

7) Estilos mínimos + foco visible (5 min)

En styles/styles.css:

```

* { box-sizing: border-box; }
body { font-family: system-ui, sans-serif; line-height: 1.5; margin: 0; }

```

```

header, main, footer { padding: 1rem; }
.skip-link {
  position: absolute; left: -999px; top: -999px;
}
.skip-link:focus {
  position: static; background: #eee; padding: .5rem;
}
nav ul { list-style: none; padding: 0; display: flex; gap: 1rem; }
a:focus { outline: 3px solid #000; }

```

8) Vista previa local (5 min)

- Abre public/index.html y ejecuta **Open with Live Server**.
- Verifica que el *skip link* funcione con **Tab** (debe aparecer y llevar al <main>).

Criterios de logro de la Hora 1:

- Node/npm y VS Code instalados.
- Proyecto con estructura básica y Git iniciado.
- Página HTML semántica mínima visible en Live Server con foco accesible.

Hora 2 (≈60 min) – Semántica + Accesibilidad aplicada

1) Añade una tabla accesible (10 min)

Debajo de “Gestión de Turnos”, inserta:

```

<table>
  <caption>Estados del sistema por entorno</caption>
  <thead>
    <tr>
      <th scope="col">Entorno</th>
      <th scope="col">Estado</th>
      <th scope="col">Última revisión</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Desarrollo</th>
      <td>Operativo</td>
      <td>2025-09-22</td>
    </tr>
    <tr>
      <th scope="row">Producción</th>
      <td>Operativo parcial</td>
      <td>2025-09-21</td>
    </tr>
  </tbody>
</table>

```

Claves: caption describe la tabla; th scope identifica encabezados.

2) Inserta una imagen con alt adecuado (5 min)

- Coloca una imagen (placeholder) en assets/images/.
- En el <article>:

```
<figure>
  
  <figcaption>Arquitectura cliente-servidor base del curso.</figcaption>
</figure>
```

3) Crea un formulario accesible (15 min)

Bajo la sección Accesibilidad:

```
<form aria-labelledby="tituloregistro">
  <h3 id="tituloregistro">Registro de participación</h3>

  <label for="nombre">Nombre completo</label>
  <input id="nombre" name="nombre" type="text" required aria-describedby="ayuda-
nombre">

  <p id="ayuda-nombre">Usa tu nombre institucional.</p>

  <label for="email">Correo</label>
  <input id="email" name="email" type="email" required aria-describedby="ayuda-email">
  <p id="ayuda-email">Formato: usuario@ejemplo.edu.ec</p>

  <fieldset>
    <legend>Preferencia de contacto</legend>
    <label><input type="radio" name="contacto" value="correo" checked> Correo</label>
    <label><input type="radio" name="contacto" value="telefono"> Teléfono</label>
  </fieldset>

  <button type="submit">Guardar</button>
</form>
```

Claves: cada input tiene label; *hints* con aria-describedby; fieldset/legend agrupan opciones.

4) Revisa jerarquía y navegación por teclado (10 min)

- Un solo <h1>; subsecciones con <h2> y contenidos con <h3>.
- Recorre con **Tab**: debe ser lógico (skip-link → nav → contenido → formulario → footer).
- Asegura foco visible (no lo elimines con CSS).

5) Validación rápida (10–15 min)

- **HTMLHint**: revisa advertencias; corrige etiquetas sin cerrar o jerarquía rota.
- **Lighthouse** (en DevTools): ejecuta auditoría de **Accessibility** (modo Desktop).
 - **Objetivo:** ≥ 90 en accesibilidad.

- Corrige: contraste, alt faltante, aria-label necesario, orden de encabezados.

Criterios de logro de la Hora 2:

- Tabla con caption y scope correctos.
- Imagen con alt significativo y figcaption.
- Formulario etiquetado, con *hints* y agrupaciones correctas.
- Auditoría inicial pasada y mejoras aplicadas.

Checklist de validación (rápido)

- ☐ Un único <h1> y jerarquía consistente.
- ☐ nav con aria-label y uso de listas.
- ☐ *Skip link* visible al enfocar.
- ☐ Imágenes con alt apropiado (o vacío si decorativas).
- ☐ Tablas con caption y scope.
- ☐ Formularios con label/aria-describedby.
- ☐ Foco siempre visible.
- ☐ Lighthouse A11y ≥ 90.

Errores comunes a evitar

- Usar <div> para botones/enlaces (usa <button>/<a>).
- Quitar el *outline* de foco sin alternativa visible.
- Encabezados salteados (pasar de h1 a h4).
- Texto en imágenes sin *overlay* que garantice contraste.
- alt genéricos (“imagen1”) o irrelevantes.

Ejercicios para resolver en clase

Ejercicio 1 — Refactor semántico

Te entregarán un HTML “anti-patrón” (solo div/span).

Tarea: Reescribelo semánticamente: introduce header, nav, main, section, article, aside, footer; corrige encabezados, agrega *skip link*, imagen con alt y una tabla con caption+scope.

Criterios de finalización:

- Estructura semántica completa.
- Navegación por teclado sin trampas.
- Lighthouse A11y ≥ 90.

Ejercicio 2 — Mini landing accesible

Crea una página con:

1. **Navegación** (3 enlaces, aria-current en la sección activa).

2. **Sección principal** con un **artículo** y una **imagen** (figure+figcaption).
3. **Tabla** con caption y cabeceras correctas.
4. **Formulario** con label/aria-describedby y agrupación (fieldset).

Criterios de finalización:

- Semántica correcta; foco visible; orden lógico de Tab.
- Contraste suficiente (usa un verificador si es necesario).
- Lighthouse A11y ≥ 90 .

Hora 1 — Entorno listo + proyecto base

1) Verifica el entorno (node -v / npm -v)

- **¿Para qué?** Comprobar que tienes el *runtime* (Node.js) y el gestor de paquetes (npm) disponibles. Si no existen o están rotos, cualquier intento de automatizar, probar o desplegar fallará.
- **Competencia:** Diagnóstico temprano del entorno y reproducibilidad (base para CI/CD y trabajo en equipo).

2) Instala Node.js LTS

- **¿Para qué?** La versión **LTS** garantiza estabilidad y soporte prolongado. Evita incompatibilidades de librerías que suelen aparecer en versiones *Current*.
- **Competencia:** Elección informada de versiones para asegurar compatibilidad de dependencias (práctica profesional).

3) Instala VS Code + extensiones

- **Live Server: ¿Para qué?** Levanta un servidor local con recarga automática. Acelera el ciclo *editar-ver*.
- **Prettier: ¿Para qué?** Formatea el código de forma consistente; reduce discusiones de estilo y errores por sangrías.
- **ESLint (para más adelante): ¿Para qué?** Detecta *smells* y errores lógicos en JS. Previene bugs en fases tempranas.
- **HTMLHint: ¿Para qué?** Valida la estructura HTML (etiquetas faltantes, jerarquías mal puestas).
- **Competencia:** Calidad de código y productividad (fundamental para equipos y escalabilidad).

4) Estructura del proyecto (public/, assets/, styles/, README.md)

- **¿Para qué?** Separar responsabilidades:
 - public/ para lo que se sirve al navegador,
 - assets/ para recursos estáticos (imágenes),
 - styles/ para el CSS,
 - README.md para documentar cómo correr/installar.
- **Competencia:** Organización profesional del repo (facilita mantenimiento, pruebas y despliegue).

5) Inicializa Git + npm

- **git init: ¿Para qué?** Versionar el proyecto (historial, ramas, *pull requests*).
- **npm init -y: ¿Para qué?** Crea package.json (metadatos, *scripts*, dependencias).
- **.gitignore: ¿Para qué?** Evitar subir archivos generados o sensibles:
 - node_modules/ (pesado y regenerable),
 - .env (secretos),
 - .DS_Store (basura de macOS).
- **Competencia:** Flujo de trabajo profesional (control de versiones + gestión de dependencias).

6) Esqueleto HTML semántico

- **¿Para qué?** Establecer una **estructura correcta** desde el día 1:
 - header/nav/main/section/article/aside/footer → semántica y SEO.
 - **Skip link** → accesibilidad por teclado.
 - aria-current="page" → indica dónde está el usuario.
 - aria-live="polite" → anuncia cambios dinámicos sin interrumpir al lector de pantalla.
- **Competencia:** Base sólida para accesibilidad (WCAG) y para integrar más adelante lógica cliente-servidor.

7) Estilos mínimos + foco visible

- **¿Para qué?** Asegurar que el **foco** sea perceptible (no romper accesibilidad). El *skip link* oculto hasta foco es un patrón estándar.
- **Competencia:** Buenas prácticas de UI accesible (requisito frecuente en proyectos reales).

8) Vista previa local (Live Server)

- **¿Para qué?** Validar rápidamente que la página carga y que la navegación por teclado funciona; detectar errores de ruta y *cache busting*.
- **Competencia:** Ciclo de feedback corto (iteración ágil).

Hora 2 — Semántica + Accesibilidad aplicada

1) Tabla accesible (caption + scope)

- **¿Para qué?** Presentar datos tabulares con **relaciones claras** entre celdas y encabezados:
 - caption = propósito de la tabla,
 - scope="col/row" = asociación para lectores de pantalla.
- **Competencia:** Comunicación de datos comprensible para todos los usuarios (base para reportes y paneles futuros).

2) Imagen con alt + figure/figcaption

- **¿Para qué?**
 - alt describe la **información** de la imagen (no su estética).
 - figcaption añade contexto.
 - Si la imagen es decorativa, alt="" para **no** distraer al lector de pantalla.
- **Competencia:** Inclusión de contenidos visuales sin barreras (norma WCAG).

3) Formulario accesible (label, aria-describedby, fieldset/legend)

- **¿Para qué?**
 - label for vincula controles con su texto.
 - aria-describedby enlaza *hints* y errores al control (lectura coherente).
 - fieldset/legend agrupa opciones relacionadas (como *radios*).
- **Competencia:** Formularios usables por teclado y tecnología asistiva (crítico en apps reales: login, registro, checkout).

4) Jerarquía y navegación por teclado

- **¿Para qué?** Garantizar flujo lógico: **un solo h1**, progresión h2→h3, y orden de *Tab* predecible.
- **Competencia:** Coherencia estructural que reduce confusión y errores de accesibilidad.

5) Validación (HTMLHint + Lighthouse)

- **¿Para qué?**
 - **HTMLHint:** detección de problemas estructurales (etiquetas mal anidadas, atributos faltantes).
 - **Lighthouse:** puntaje y recomendaciones de **Accesibilidad** (contraste, *alt*, roles/labels).
- **Competencia:** Cultura de **calidad** basada en herramientas (medir, corregir, volver a medir).

Checklist de validación (¿para qué sirve?)

- **Propósito:** Lista de “salida de calidad” antes de dar por terminada la práctica. Minimiza retrabajo y asegura estándares mínimos (p. ej., Lighthouse ≥ 90 en A11y).

Errores comunes (¿por qué son problemáticos?)

- **<div> como botón/enlace:** rompe semántica, atajos y lectores de pantalla no entienden su rol.
- **Quitar el *outline* de foco:** deja “ciegos” a usuarios de teclado.
- **Saltar niveles de encabezado:** desordena la estructura y confunde a tecnologías asistivas.
- **Texto sobre imágenes sin contraste:** ilegible para varios usuarios y penalizado en auditorías.
- **alt genéricos:** no aportan información; lectores de pantalla leen algo inútil.

Ejercicios (¿qué competencia desarrolla cada uno?)

Ejercicio 1 — Refactor semántico

- **¿Para qué?** Convertir un HTML “plano” en uno **correctamente estructurado** y accesible.
- **Competencia:** Diagnóstico y mejora de código heredado (caso real en empresas).
- **Relación RDA/retos:** cimenta el RDA 1 (fundamentos) y prepara al estudiante para integrar estructuras y UI más adelante.

Ejercicio 2 — Mini landing accesible

- **¿Para qué?** Ensayar un *layout* completo con navegación, contenido, datos y formulario, **todo accesible**.
- **Competencia:** Composición integral de una página lista para evolucionar hacia cliente-servidor (semana 4+).
- **Relación RDA/retos:** provee el esqueleto sobre el cual, en semanas posteriores, se integrarán lógicas (colas, pilas, listas) y seguridad.

Semana 2 – Guía de prácticas (2h)

Objetivo: Diseñar y validar un formulario accesible con semántica correcta (landmarks), ayudas y errores anunciables, validación nativa y reglas propias (contraseñas, fechas, email “ya existente”), y verificación rápida WCAG.

Requisitos previos: VS Code + Live Server (Semana 1), nociones básicas de HTML/CSS/JS.

Estructura del proyecto

```
semana2-formularios/  
├── public/  
│   └── index.html  
├── styles/  
│   └── styles.css  
└── scripts/  
    └── app.js
```

Abre la carpeta en VS Code. Usa Live Server sobre public/index.html.

Hora 1 (≈60 min) — Maquetado semántico + validación nativa

1) Crea public/index.html

```
<!doctype html>  
<html lang="es">  
<head>  
  <meta charset="utf-8" />  
  <title>Registro de Cuenta – Semana 2 (Hotfix)</title>  
  <meta name="viewport" content="width=device-width, initial-scale=1" />  
  <meta name="description" content="Formulario accesible con validación nativa y personalizada." />  
  <link rel="stylesheet" href="../styles/styles.css" />  
</head>  
<body>  
  <!-- Skip link para teclado/lectores -->  
  <a class="skip-link" href="#contenido">Saltar al contenido</a>  
  
  <header>  
    <h1>Plataforma Académica</h1>  
    <nav aria-label="Principal">  
      <ul>  
        <li><a href="#" aria-current="page">Inicio</a></li>  
        <li><a href="#formulario">Registro</a></li>  
      </ul>  
    </nav>  
  </header>  
  
  <main id="contenido">  
    <!-- Resumen de errores (JS lo rellena si hace falta) -->
```

```

<section aria-labelledby="t-resumen" class="visually-group" id="resumen-seccion">
  <h2 id="t-resumen">Resumen de errores</h2>
  <div id="form-summary" role="alert" aria-live="polite"></div>
</section>

<section id="formulario" aria-labelledby="t-form">
  <h2 id="t-form">Registro de cuenta</h2>

  <!-- Usamos novalidate para controlar mensajes,
    pero dejamos atributos nativos en el resto de campos -->
  <form id="f-registro" novalidate aria-describedby="ayuda-general">
    <p id="ayuda-general">Completa los campos obligatorios. Navega con Tab y Enter.</p>

    <!-- Datos personales -->
    <fieldset>
      <legend>Datos personales</legend>

      <label for="nombre">Nombre</label>
      <input id="nombre" name="nombre" type="text" required minlength="2"
        autocomplete="given-name" aria-describedby="hint-nombre nombre-err">
      <small id="hint-nombre">Tal como figura en tu documento (mínimo 2 caracteres).</small>
      <span id="nombre-err" class="error" role="alert" hidden></span>

      <label for="apellidos">Apellidos</label>
      <input id="apellidos" name="apellidos" type="text" required minlength="2"
        autocomplete="family-name" aria-describedby="hint-apellidos apellidos-err">
      <small id="hint-apellidos">Mínimo 2 caracteres.</small>
      <span id="apellidos-err" class="error" role="alert" hidden></span>

      <label for="email">Correo institucional</label>
      <input id="email" name="email" type="email" required
        autocomplete="email" aria-describedby="hint-email email-status email-err">
      <small id="hint-email">Formato: usuario@dominio.</small>
      <!-- Estado no intrusivo (validación asíncrona simulada) -->
      <small id="email-status" role="status" aria-live="polite"></small>
      <span id="email-err" class="error" role="alert" hidden></span>

      <label for="tel">Teléfono</label>
      <input id="tel" name="tel" type="tel" inputmode="tel"
        autocomplete="tel" aria-describedby="hint-tel">
      <small id="hint-tel">Incluye código de área si aplica.</small>
    </fieldset>

    <!-- Acceso -->
    <fieldset>
      <legend>Acceso</legend>

      <label for="pass">Contraseña</label>
      <!-- SIN minlength: la longitud se valida en JS -->

```

```

        <input id="pass" name="pass" type="password" required
            aria-describedby="hint-pass pass-err pass-strength-text">
        <small id="hint-pass">Mínimo 8 caracteres. Combina mayúsculas, números y
símbolos.</small>
        <progress id="pass-strength" value="0" max="4" aria-hidden="true"></progress>
        <small id="pass-strength-text" role="status" aria-live="polite"></small>
        <span id="pass-err" class="error" role="alert" hidden></span>

        <label for="pass2">Confirmar contraseña</label>
        <input id="pass2" name="pass2" type="password" required
            aria-describedby="pass2-err">
        <span id="pass2-err" class="error" role="alert" hidden></span>
    </fieldset>

    <!-- Preferencias -->
    <fieldset>
        <legend>Preferencias</legend>
        <label><input type="radio" name="contacto" value="correo" checked> Contacto por
correo</label>
        <label><input type="radio" name="contacto" value="telefono"> Contacto por
teléfono</label>

        <label><input type="checkbox" id="news" name="news" value="si"> Deseo recibir
boletines</label>
    </fieldset>

    <!-- Aceptación -->
    <fieldset>
        <legend>Condiciones</legend>
        <label>
            <input type="checkbox" id="acepto" name="acepto" required aria-describedby="acepto-
err">

            Acepto términos y condiciones
        </label>
        <span id="acepto-err" class="error" role="alert" hidden></span>
    </fieldset>

    <button type="submit">Crear cuenta</button>
</form>
</section>
</main>

<footer>
    <p>© 2025. Curso Desarrollo en Plataformas.</p>
</footer>

<!-- cache-busting para evitar JS viejo en caché -->
<script src="../scripts/app.js?v=hotfix5"></script>
</body>

```

```
</html>
```

2) Crea styles/styles.css

```
/* Base y foco visible */
* { box-sizing: border-box; }
body { font-family: system-ui, -apple-system, Segoe UI, Roboto, sans-serif; line-height: 1.5;
margin: 0; }
header, main, footer { padding: 1rem; }
:focus-visible { outline: 3px solid #000; outline-offset: 2px; }

/* Skip link accesible */
.skip-link { position: absolute; left: -999px; top: -999px; }
.skip-link:focus { position: static; background:#eee; padding:.5rem; }

/* Navegación simple */
nav ul { list-style: none; padding: 0; display: flex; gap: 1rem; }

/* Formulario */
form { max-width: 780px; }
fieldset { border: 1px solid #ddd; padding: 1rem; margin-bottom: 1rem; border-radius: .5rem; }
legend { font-weight: 600; }
label { display: block; margin-top: .75rem; }
input, select, textarea { width: 100%; padding: .6rem; margin-top: .25rem; border-radius: .4rem;
border: 1px solid #bbb; }
button { margin-top: 1rem; padding: .7rem 1rem; border-radius: .5rem; border: 1px solid #333;
background: #fff; cursor: pointer; }

/* Estados nativos (siguen aplicando a otros campos) */
input:required:invalid, select:required:invalid { border-color: #c00; }
input:required:valid, select:required:valid { border-color: #0a0; }

/* Mensajes */
.error[hidden] { display: none; }
.error { color: #a40000; margin-top: .25rem; }

/* Resumen de errores */
.visually-group { margin-bottom: 1rem; }
#form-summary p { margin: 0 0 .25rem 0; }
#form-summary ul { margin: .25rem 0 0 1rem; }
#form-summary a { text-decoration: underline; }

/* Accesibilidad visual extra */
@media (prefers-reduced-motion: reduce){
  * { animation-duration: .01ms !important; animation-iteration-count: 1 !important; }
}
```

3) Verifica en Live Server

- Recorre con Tab/Shift+Tab: foco visible, orden lógico.
- Prueba validación nativa: deja vacíos obligatorios, prueba email mal formado.

Hora 2 (≈60 min) — Validación personalizada (JS) + accesibilidad

4) Crea scripts/app.js

```
// ===== Identificador de versión (para confirmar que cargó este archivo) =====
console.log('%cSemana2 app.js HOTFIX v5', 'color:green', new Date().toISOString());

// ===== Utilidades de mensajes =====
function plural(n, uno, otros){ return n === 1 ? uno : otros; }

const messages = {
  valueMissing: 'Completa este campo.',
  typeMismatch: 'Revisa el formato (p. ej., usuario@dominio).',
  patternMismatch: 'Sigue el formato indicado.',
  tooShort: el => {
    const rest = el.minLength - el.value.length;
    return `Añade ${rest} ${plural(rest, 'carácter', 'caracteres')} más.`;
  },
  tooLong: el => `Reduce a ${el.maxLength} caracteres.`,
  rangeUnderflow: el => `El valor mínimo es ${el.min}.`,
  rangeOverflow: el => `El valor máximo es ${el.max}.`,
  stepMismatch: 'Ajusta al paso permitido.',
  badInput: 'Usa solo números en este campo.',
  customError: el => el.validationMessage || 'Revisa este dato.'
};

function nativeMessage(el){
  const v = el.validity;
  if (v.valid) return '';
  for (const k of Object.keys(messages)){
    if (v[k]){
      const m = messages[k];
      return typeof m === 'function' ? m(el) : m;
    }
  }
  return 'Revisa este dato.';
}

function showError(el, msg){
  const ids = (el.getAttribute('aria-describedby')||'').split(' ');
  const errId = ids.find(id => id && id.endsWith('-err'));
  const errEl = errId ? document.getElementById(errId) : null;

  el.setCustomValidity(msg||'');
  if (msg){
    el.setAttribute('aria-invalid', 'true');
    if (errEl){ errEl.textContent = msg; errEl.hidden = false; }
  } else {
    el.removeAttribute('aria-invalid');
  }
}
```

```

    if (errEl){ errEl.hidden = true; errEl.textContent = ''; }
  }
}

function renderSummary(form, invalidFields){
  const box = document.getElementById('form-summary');
  if (!invalidFields.length){ box.innerHTML = ''; return; }
  const items = invalidFields.map(({el, msg})=>{
    const label = form.querySelector(`label[for="${el.id}"]`);
    const texto = label ? label.textContent.trim() : el.name || el.id;
    return `<li><a href="#${el.id}">${texto}</a> ${msg}</li>`;
  }).join('');
  box.innerHTML = `<p>Por favor, corrige lo siguiente:</p><ul>${items}</ul>`;
}

function passwordStrength(value, minLen){
  let s = 0;
  if (value.length >= minLen) s++;
  if (/[A-Z]/.test(value)) s++;
  if (/\\d/.test(value)) s++;
  if (/^[A-Za-z0-9]/.test(value)) s++;
  return s; // 0..4
}

document.addEventListener('DOMContentLoaded', () => {
  const form      = document.getElementById('f-registro');
  const fields     = form.querySelectorAll('input, select, textarea');

  const email      = document.getElementById('email');
  const emailStatus= document.getElementById('email-status');
  let emailTimer = 0, emailQuery = 0;

  const pass       = document.getElementById('pass');
  const pass2      = document.getElementById('pass2');
  const passMeter  = document.getElementById('pass-strength');
  const passText   = document.getElementById('pass-strength-text');
  const passHint   = document.getElementById('hint-pass');

  // ===== Config global de contraseña =====
  const DESIRED_MIN = 8; // cambia a 12/16 si quieres
  // Quita cualquier minlength que venga del HTML o de otro script
  if (pass.hasAttribute('minlength')) pass.removeAttribute('minlength');
  // Refleja el mínimo en el hint
  if (passHint) passHint.textContent =
    `Mínimo ${DESIRED_MIN} caracteres. Combina mayúsculas, números y símbolos.`;

  // ===== Validación nativa para todo MENOS 'pass' (que controlamos nosotros) =====
  fields.forEach(el => {
    if (el.id === 'pass') return;

```

```

    el.addEventListener('input', () => showError(el, nativeMessage(el)));
    el.addEventListener('blur', () => showError(el, nativeMessage(el)));
  });

  // ===== Regla de longitud propia para 'pass' =====
  function checkPasswordMin(){
    const rest = DESIRED_MIN - pass.value.length;
    if (rest > 0){
      const msg = `Mínimo ${DESIRED_MIN} caracteres. Añade ${rest}
${plural(rest, 'carácter', 'caracteres')}`;
      pass.setCustomValidity(msg);
      showError(pass, msg);
    } else {
      pass.setCustomValidity('');
      showError(pass, '');
    }
  }

  // Fuerza y mensajes accesibles
  pass.addEventListener('input', () => {
    checkPasswordMin(); // primero nuestra regla
    const score = passwordStrength(pass.value, DESIRED_MIN);
    passMeter.value = score;
    const textos = ['Muy débil', 'Débil', 'Aceptable', 'Buena', 'Fuerte'];
    passText.textContent = pass.value ? `Fuerza: ${textos[score]}` : '';
  });
  pass.addEventListener('blur', checkPasswordMin);

  // Confirmación de contraseña
  function checkPasswords(){
    pass2.setCustomValidity(''); showError(pass2, '');
    if (pass.value && pass2.value && pass.value !== pass2.value){
      const msg = 'La confirmación no coincide con la contraseña.';
      pass2.setCustomValidity(msg); showError(pass2, msg);
    }
  }
  pass.addEventListener('input', checkPasswords);
  pass2.addEventListener('input', checkPasswords);

  // Validación asíncrona simulada para correo (debounce 300 ms)
  email.addEventListener('input', () => {
    clearTimeout(emailTimer);
    emailStatus.textContent = '';
    if (!email.checkValidity()) return;

    emailTimer = setTimeout(() => {
      const qid = ++emailQuery;
      emailStatus.textContent = 'Comprobando disponibilidad...';

```

```

// Simulación: "existe" si incluye "test"
const exists = /test/i.test(email.value);

setTimeout(() => {
  if (qid !== emailQuery) return; // respuesta vieja
  if (exists){
    const msg = 'Este correo ya está registrado. ¿Quieres iniciar sesión?';
    email.setCustomValidity(msg);
    showError(email, msg);
    emailStatus.textContent = 'Ya registrado.';
  } else {
    email.setCustomValidity('');
    showError(email, '');
    emailStatus.textContent = 'Disponible.';
  }
}, 300);
}, 300);
});

// Envío: muestra resumen y enfoca el primer error
form.addEventListener('submit', (e) => {
  const invalid = [];

  // Revisa todos menos 'pass' con nativo
  for (const el of fields){
    if (el.id === 'pass') continue;
    const msg = nativeMessage(el);
    showError(el, msg);
    if (msg) invalid.push({el, msg});
  }

  // Reglas propias
  checkPasswordMin();
  checkPasswords();
  if (!pass.validity.valid) invalid.push({el: pass, msg: pass.validationMessage});
  if (!pass2.validity.valid) invalid.push({el: pass2, msg: pass2.validationMessage});

  if (invalid.length){
    e.preventDefault();
    renderSummary(form, invalid);
    invalid[0].el.focus();
  } else {
    renderSummary(form, []);
    // aquí enviaría normalmente
  }
});
});

```

5) Pruebas rápidas

- **Teclado:** completa el flujo sin mouse; el foco debe ser claro y el orden lógico.
- **Errores:** fuerza 3–4 errores; deben aparecer junto a los campos y además en el **resumen** superior, con anclas que te llevan al campo.
- **Nativo:** email mal formado → borde rojo por estados :invalid.
- **JS:** contraseñas distintas → error en confirmación + resumen.
- **Asíncrono (simulado):** escribe un correo con “test” → “ya registrado” + bloqueo del envío.
- **Accesibilidad:** el resumen tiene role="alert", el estado del email usa role="status", y el foco salta al **primer** error al enviar.

Ejercicios para resolver en clase (15–20 min)

Ejercicio 1 — Campo “Identificación” con patrón local

Añade un campo “Identificación nacional (10 dígitos)” antes del teléfono:

- `type="text", pattern="^\d{10}$", inputmode="numeric".`
- Hint con ejemplo y error vinculado; integra el campo a la validación (verás el mensaje nativo + el personalizado si quieres).

Criterio de finalización: el error aparece junto al campo y en el resumen; el envío se bloquea si no cumple.

Ejercicio 2 — Select de provincia + código postal

Agrega select de provincia (3–5 opciones) y un input postal-code (`minlength="4" maxlength="6", inputmode="numeric"`).

- Usa tokens de autocomplete (`address-level1, postal-code`).
- Añade validación nativa y mensajes claros.

Criterio de finalización: con valores inválidos, se muestran errores vinculados y el resumen; con datos válidos, el formulario envía.

Guía de Prácticas (2h) – Semana 3

Temas aplicados: CSS3 y diseño adaptable · Box Model · Flexbox · Grid · Media Queries · Frameworks (Bootstrap, Tailwind)

1) Objetivos de la práctica

- Comprender y aplicar el **Box Model** para estructurar cajas.
- Construir layouts con **Flexbox y Grid**.
- Usar **Media Queries** para hacer el diseño responsivo.
- Comparar el desarrollo con **CSS puro** frente a frameworks (Bootstrap y Tailwind).

2) Cronograma (120 min)

Bloque	Tiempo	Actividad
A	10 min	Preparar entorno de trabajo
B	30 min	Ejercicio 1: Box Model y tipografía fluida
C	40 min	Ejercicio 2: Layout con Flexbox y Grid
D	25 min	Ejercicio 3: Media Queries (tablet/escritorio)
E	10 min	Ejercicio 4: Comparación con frameworks

3) Ejercicio 1 — Box Model + Tipografía fluida (sin Flex/Grid todavía)

Meta: entender cómo se calculan espacios (content, padding, border, margin) y cómo usar tipografía fluida con clamp().

Pasos:

1. Define box-sizing: border-box; para todas las cajas.
2. Usa padding y margin para espaciar contenido y separar bloques.
3. Implementa tipografía fluida con clamp().

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Semana 3 · Ejercicio 1 — Box Model + Tipografía fluida</title>
  <style>
    /* ===== Reset mínimo + Box Model coherente ===== */
    * { box-sizing: border-box; margin: 0; padding: 0; }

    /* ===== Tipografía base ===== */
    html { font-size: 100%; } /* 16px por defecto en la mayoría de navegadores */
    body {
      font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
      line-height: 1.5;
      color: #111;
      background: #fff;
    }

    /* ===== Tokens simples (espacios y contenedor) ===== */
    :root{
      --space-1: .5rem; --space-2: 1rem; --space-3: 1.5rem; --space-4: 2rem;
```

```

--maxw: 64rem; /* ~1024px */
--border: 1px solid #e5e7eb;
}
.wrap { max-width: var(--maxw); margin-inline: auto; padding: var(--space-2); }

/* ===== Encabezado (sin flex aún) ===== */
header { border-bottom: var(--border); padding: var(--space-2) 0; }
.brand { font-weight: 700; font-size: 1.125rem; margin-bottom: .25rem; }
nav a {
  display: inline-block;
  text-decoration: none;
  color: #0366d6;
  margin-right: 1rem;
  padding: .25rem 0;
}

/* ===== Card principal (en columna) ===== */
.card {
  background: #fff;
  border: var(--border);
  border-radius: .5rem;
  padding: var(--space-3);
  margin-block: var(--space-3);
  /* Box Model aplicado: padding para que "respire" internamente */
}
.card h1 {
  /* Tipografía fluida: mínimo 1.5rem, máximo 2.25rem, zona fluida 2.5vw */
  font-size: clamp(1.5rem, 2.5vw, 2.25rem);
  margin-bottom: .5rem;
}
.card p { margin-bottom: var(--space-2); }
.card img {
  /* Medios fluidos: nunca sobrepasan el ancho de su contenedor */
  max-width: 100%; height: auto; border-radius: .5rem; border: var(--border);
}

/* ===== Features (bloques apilados por ahora) ===== */
.features { margin-block: var(--space-3); }
.feature {
  background: #f9fafb;
  border: var(--border);
  border-radius: .5rem;
  padding: var(--space-3);
  font-weight: 600;
  text-align: center;
  /* Márgenes externos: separa esta caja de la siguiente (Box Model) */
  margin-bottom: var(--space-2);
}

/* ===== Pie ===== */
footer { border-top: var(--border); text-align: center; padding: var(--space-2) 0; }
</style>
</head>
<body>
<header>
  <div class="wrap">
    <div class="brand">MiSitio</div>
    <nav aria-label="principal">
      <a href="#">Inicio</a>
      <a href="#">Blog</a>
      <a href="#">Contacto</a>
    </nav>
  </div>

```

```

</header>

<main class="wrap">
  <article class="card">
    <h1>Bienvenido</h1>
    <p>Este es el contenido principal. Aquí practicamos <strong>Box Model</strong> (padding, border, margin) y
    <strong>tipografía fluida</strong> con <code>clamp()</code>.</p>
    
  </article>

  <section class="features" aria-label="Módulos">
    <article class="feature">Módulo A</article>
    <article class="feature">Módulo B</article>
    <article class="feature">Módulo C</article>
  </section>
</main>

<footer>
  <div class="wrap"><small>© 2025 — Ejercicio 1 (CSS puro sin Flex/Grid)</small></div>
</footer>
</body>
</html>

```

4) Layout con Flexbox (header) + Grid (card/features), mobile-first (sin Media Queries aún)

Meta: organizar la página combinando Flexbox para cabecera y Grid para el contenido.

Pasos:

1. Aplica display: flex en el header para alinear logo y menú.
2. Convierte .card en grid: 1 columna en móvil, 2 columnas en pantallas más grandes.
3. Configura .features como grid que irá cambiando con media queries.

```

<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Semana 3 · Ejercicio 2 — Flexbox + Grid (mobile-first)</title>
  <style>
    /* Reset + box model */
    * { box-sizing: border-box; margin: 0; padding: 0; }

    /* Tipografía base */
    body {
      font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
      line-height: 1.5; color: #111; background: #fff;
    }

    /* Tokens */
    :root{
      --space-1: .5rem; --space-2: 1rem; --space-3: 1.5rem; --space-4: 2rem;
      --maxw: 72rem; --border: 1px solid #e5e7eb;
    }
    .wrap { max-width: var(--maxw); margin-inline: auto; padding: var(--space-2); }

    /* ===== Header con Flexbox (alineación en 1 eje) ===== */
    header { border-bottom: var(--border); }
    .header-inner {
      display: flex; align-items: center; justify-content: space-between;

```

```

    gap: var(--space-2); padding: var(--space-2) 0;
  }
.brand { font-weight: 700; font-size: 1.125rem; }
nav a { text-decoration: none; color: #0366d6; margin-left: 1rem; }

/* ===== Card con Grid (macro) — por ahora 1 columna ===== */
.card {
  display: grid; /* activamos grid */
  gap: var(--space-2); /* separación entre celdas */
  background: #fff; border: var(--border); border-radius: .5rem;
  padding: var(--space-3); margin-block: var(--space-3);
}
.card h1 { font-size: clamp(1.5rem, 2.5vw, 2.25rem); }
.card img { max-width: 100%; height: auto; border-radius: .5rem; border: var(--border); }

/* ===== Features con Grid (macro) — 1 columna por ahora ===== */
.features { display: grid; gap: var(--space-2); margin-block: var(--space-3); }
.feature {
  background: #f9fafb; border: var(--border); border-radius: .5rem;
  padding: var(--space-3); text-align: center; font-weight: 600;
}

/* Pie */
footer { border-top: var(--border); text-align: center; padding: var(--space-2) 0; }
</style>
</head>
<body>
  <header>
    <div class="wrap header-inner">
      <div class="brand">MiSitio</div>
      <nav aria-label="principal">
        <a href="#">Inicio</a>
        <a href="#">Blog</a>
        <a href="#">Contacto</a>
      </nav>
    </div>
  </header>

  <main class="wrap">
    <article class="card">
      <!-- En móvil: 1 columna (texto sobre imagen) -->
      <div class="card-text">
        <h1>Bienvenido</h1>
        <p>Ahora combinamos <strong>Flexbox</strong> (en el header) y <strong>Grid</strong> (en esta tarjeta y en
features), todavía sin Media Queries.</p>
      </div>
      
    </article>

    <section class="features" aria-label="Módulos">
      <article class="feature">Módulo A</article>
      <article class="feature">Módulo B</article>
      <article class="feature">Módulo C</article>
    </section>
  </main>

  <footer>
    <div class="wrap"><small>© 2025 — Ejercicio 2 (Flexbox + Grid, sin MQ)</small></div>
  </footer>
</body>
</html>

```

5) Ejercicio 3 – Media Queries (tablet/escritorio) sobre el layout de Flex+Grid

Meta: aplicar breakpoints para reorganizar el layout en tablet y escritorio.

Código:

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Semana 3 · Ejercicio 3 — Media Queries (tablet/escritorio)</title>
  <style>
    /* Reset + box model */
    * { box-sizing: border-box; margin: 0; padding: 0; }

    /* Tipografía base */
    body {
      font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
      line-height: 1.5; color: #111; background: #fff;
    }

    /* Tokens */
    :root{
      --space-1: .5rem; --space-2: 1rem; --space-3: 1.5rem; --space-4: 2rem;
      --maxw: 72rem; --border: 1px solid #e5e7eb;
    }
    .wrap { max-width: var(--maxw); margin-inline: auto; padding: var(--space-2); }

    /* ===== Header con Flexbox ===== */
    header { border-bottom: var(--border); }
    .header-inner {
      display: flex; align-items: center; justify-content: space-between;
      gap: var(--space-2); padding: var(--space-2) 0;
    }
    .brand { font-weight: 700; font-size: 1.125rem; }
    nav a { text-decoration: none; color: #0366d6; margin-left: 1rem; }

    /* ===== Card con Grid ===== */
    .card {
      display: grid; gap: var(--space-2);
      background: #fff; border: var(--border); border-radius: .5rem;
      padding: var(--space-3); margin-block: var(--space-3);
    }
    .card h1 { font-size: clamp(1.5rem, 2.5vw, 2.25rem); }
    .card img { max-width: 100%; height: auto; border-radius: .5rem; border: var(--border); }

    /* ===== Features con Grid ===== */
    .features { display: grid; gap: var(--space-2); margin-block: var(--space-3); }
    .feature {
      background: #f9fafb; border: var(--border); border-radius: .5rem;
      padding: var(--space-3); text-align: center; font-weight: 600;
    }

    /* ===== Pie ===== */
    footer { border-top: var(--border); text-align: center; padding: var(--space-2) 0; }

    /* ===== MEDIA QUERIES (Mobile-first) ===== */
    /* Tablet: a partir de 768px */
    @media (min-width: 768px) {
      .card {
        /* 2 columnas: texto (más ancho) + imagen */
      }
    }
  </style>
</head>
<body>
  <div class="wrap">
    <div class="header">
      <div class="header-inner">
        <div class="brand">
          <h1>
            </h1>
        </div>
        <div class="nav">
          <a href="#">Inicio</a>
          <a href="#">Proyectos</a>
          <a href="#">Contacto</a>
        </div>
      </div>
    </div>
    <div class="card">
      <h1>
        </h1>
    </div>
    <div class="features">
      <div class="feature">
        <h2>
          </h2>
      </div>
      <div class="feature">
        <h2>
          </h2>
      </div>
      <div class="feature">
        <h2>
          </h2>
      </div>
    </div>
    <div class="card">
      <h1>
        </h1>
    </div>
  </div>
  <div class="wrap">
    <div class="footer">
      <p>
        </p>
    </div>
  </div>
</body>
</html>
```

```

    grid-template-columns: 1.25fr 1fr;
    align-items: start;
  }
  .features { grid-template-columns: repeat(2, 1fr); }
  /* (Opcional) crecer la marca */
  .brand { font-size: 1.25rem; }
}

/* Escritorio: a partir de 1200px */
@media (min-width: 1200px) {
  .features { grid-template-columns: repeat(3, 1fr); }
  .wrap { padding: var(--space-3); }
}

/* Preferencias de usuario (accesibilidad) */
@media (prefers-reduced-motion: reduce) {
  * { animation: none !important; transition-duration: 0.01ms !important; }
}
@media (prefers-color-scheme: dark) {
  body { background: #0b0d10; color: #e5e7eb; }
  .feature, .card { background: #111827; border-color: #1f2937; }
  nav a { color: #93c5fd; }
}
</style>
</head>
<body>
  <header>
    <div class="wrap header-inner">
      <div class="brand">MiSitio</div>
      <nav aria-label="principal">
        <a href="#">Inicio</a>
        <a href="#">Blog</a>
        <a href="#">Contacto</a>
      </nav>
    </div>
  </header>

  <main class="wrap">
    <article class="card">
      <div class="card-text">
        <h1>Bienvenido</h1>
        <p>Aplicamos <strong>Media Queries</strong>: en tablet la tarjeta pasa a 2 columnas; las <em>features</em>
a 2 columnas; en escritorio, a 3.</p>
      </div>
      
    </article>

    <section class="features" aria-label="Módulos">
      <article class="feature">Módulo A</article>
      <article class="feature">Módulo B</article>
      <article class="feature">Módulo C</article>
    </section>
  </main>

  <footer>
    <div class="wrap"><small>© 2025 — Ejercicio 3 (Flex+Grid + Media Queries)</small></div>
  </footer>
</body>
</html>

```

👉 Resultado esperado:

- En móvil: todo en una sola columna.
- En tablet: .card a 2 columnas, .features a 2 columnas.
- En escritorio: .features a 3 columnas.

6) Ejercicio 4 – Comparación con Frameworks (10 min)

Meta: construir el mismo layout en tres enfoques y comparar resultados.

a) Versión con CSS puro

index.html

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Práctica CSS puro - Semana 3</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header class="site-header">
    <div class="brand">MiSitio</div>
    <nav class="site-nav">
      <a href="#">Inicio</a>
      <a href="#">Blog</a>
      <a href="#">Contacto</a>
    </nav>
  </header>

  <main class="content">
    <article class="card">
      <div class="card-text">
        <h1>Bienvenido</h1>
        <p>Este es el contenido principal. Ajusta estilos y layout según el ancho de pantalla.</p>
      </div>
      
    </article>

    <section class="features">
      <article class="feature">Módulo A</article>
      <article class="feature">Módulo B</article>
      <article class="feature">Módulo C</article>
    </section>
  </main>

  <footer class="site-footer">
    <small>© 2025 — CSS puro</small>
  </footer>
</body>
</html>
```

styles.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: sans-serif; line-height: 1.5; }

.content { max-width: 1200px; margin: auto; padding: 1rem; }
```

```

.site-header { display: flex; justify-content: space-between; align-items: center; padding: 1rem; border-bottom: 1px solid #ccc; }
.site-nav a { margin-left: 1rem; text-decoration: none; color: #0366d6; }

.card { display: grid; gap: 1rem; margin: 2rem 0; }
.card img { width: 100%; border-radius: 8px; }
.card-text h1 { font-size: clamp(1.5rem, 2.5vw, 2.25rem); }

.features { display: grid; gap: 1rem; }
.feature { background: #f4f4f4; padding: 1rem; border-radius: 8px; border: 1px solid #ddd; text-align: center; }

.site-footer { text-align: center; padding: 1rem; border-top: 1px solid #ccc; margin-top: 2rem; }

@media (min-width: 768px) {
  .card { grid-template-columns: 2fr 1fr; }
  .features { grid-template-columns: repeat(2, 1fr); }
}
@media (min-width: 1200px) {
  .features { grid-template-columns: repeat(3, 1fr); }
}

```

b) Versión con Bootstrap 5

index.html

```

<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Práctica Bootstrap - Semana 3</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
  <header class="bg-light border-bottom mb-4">
    <div class="container d-flex justify-content-between align-items-center py-2">
      <span class="fw-bold">MiSitio</span>
      <nav>
        <a class="mx-2 text-decoration-none text-primary" href="#">Inicio</a>
        <a class="mx-2 text-decoration-none text-primary" href="#">Blog</a>
        <a class="mx-2 text-decoration-none text-primary" href="#">Contacto</a>
      </nav>
    </div>
  </header>

  <main class="container mb-4">
    <div class="row g-3 align-items-center mb-4">
      <div class="col-md-8">
        <h1 class="fs-2">Bienvenido</h1>
        <p>Este es el contenido principal. Ajusta estilos y layout según el ancho de pantalla.</p>
      </div>
      <div class="col-md-4">
        
      </div>
    </div>

    <div class="row g-3">
      <div class="col-12 col-md-6 col-xl-4"><div class="p-3 bg-light border rounded text-center">Módulo
A</div></div>
      <div class="col-12 col-md-6 col-xl-4"><div class="p-3 bg-light border rounded text-center">Módulo
B</div></div>
    </div>
  </main>

```

```

    <div class="col-12 col-md-6 col-xl-4"><div class="p-3 bg-light border rounded text-center">Módulo
</div></div>
  </div>
</main>

<footer class="bg-light border-top text-center py-3">
  <small>© 2025 — Bootstrap</small>
</footer>
</body>
</html>

```

c) Versión con Tailwind CSS

index.html

```

<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Práctica Tailwind - Semana 3</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="font-sans text-gray-900">
  <header class="flex justify-between items-center border-b p-4 bg-white">
    <span class="font-bold">MiSitio</span>
    <nav class="space-x-4">
      <a href="#" class="text-blue-600 hover:underline">Inicio</a>
      <a href="#" class="text-blue-600 hover:underline">Blog</a>
      <a href="#" class="text-blue-600 hover:underline">Contacto</a>
    </nav>
  </header>

  <main class="max-w-5xl mx-auto p-4">
    <div class="grid gap-4 md:grid-cols-2 items-center mb-6">
      <div>
        <h1 class="text-2xl md:text-3xl font-bold mb-2">Bienvenido</h1>
        <p class="text-gray-700">Este es el contenido principal. Ajusta estilos y layout según el ancho de pantalla.</p>
      </div>
      
    </div>

    <section class="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
      <div class="bg-gray-100 border rounded-lg p-6 text-center font-semibold">Módulo A</div>
      <div class="bg-gray-100 border rounded-lg p-6 text-center font-semibold">Módulo B</div>
      <div class="bg-gray-100 border rounded-lg p-6 text-center font-semibold">Módulo C</div>
    </section>
  </main>

  <footer class="border-t text-center py-3 mt-6 bg-white">
    <small>© 2025 — Tailwind</small>
  </footer>
</body>
</html>

```